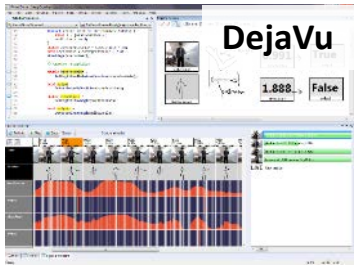
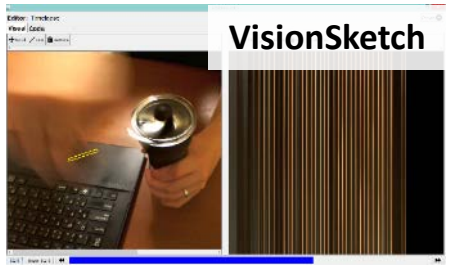
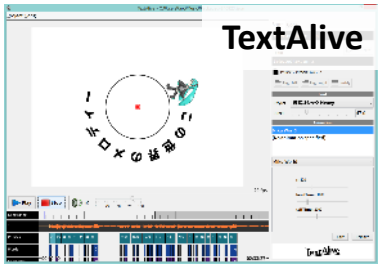
実世界Live Programmingの実現に向けて









加藤 淳 http://junkato.jp/ja/

産業技術総合研究所 情報技術研究部門
メディアインタラクション研究グループ

加藤 淳（かとう じゅん） http://junkato.jp/ja/

研究テーマ

Human-Computer Interaction（人とコンピュータの関係改善）

とくに、**統合開発環境**を使いやすくする研究を通して
より多くの人が快適にプログラミングできるようにしてきた

経歴

- **JST ERATO** 五十嵐プロジェクト 2008.4-2013.2（学部4年～）
- **東京大学** 五十嵐健夫研究室 修士・博士 2009.4-2014.3
- **Microsoft Research** Asia/Redmondインターン 2012.1-4,6-9
- **Adobe Research** Seattle インターン 2013.8-11
- **産業技術総合研究所** 研究員 2014.4-

研究プロジェクト一覧

2008 2009 2010 2011 2012 2013-

Multi-touch Multi-robot



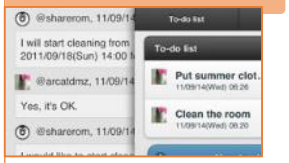
ACM CHI'09

Surfboard



ACM UIST'10

Sharedo



WISS'11

DejaVu



ACM UIST'12

VisionSketch



GI'14

CRISTAL



ACM SIGGRAPH'09

It's Alive!



ACM PLDI'13

Pressing



ACM UIST'09

IntelliViewer



ACM UIST'10

Picode



ACM CHI'13

Phybots



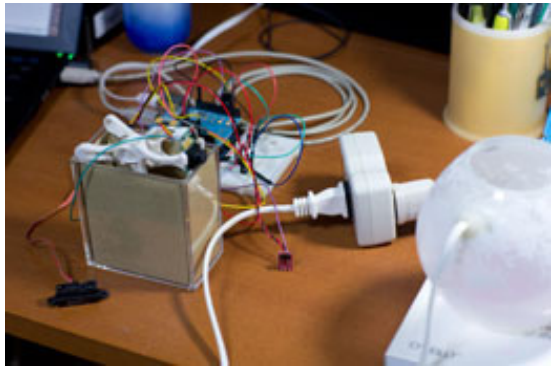
ACM DIS'12

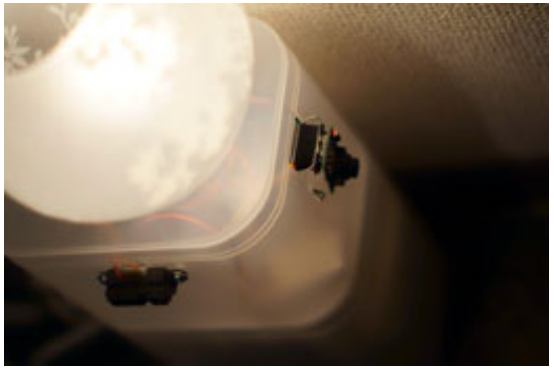
統合開発環境の研究

本日の講演内容

- 実世界Programming
  - 楽しい
  - 難しい
  - 想像力
- Live Programming
  - 死んでる
  - 生かす
  - 全部生かすと壊しやすい

実世界Programmingは楽しい











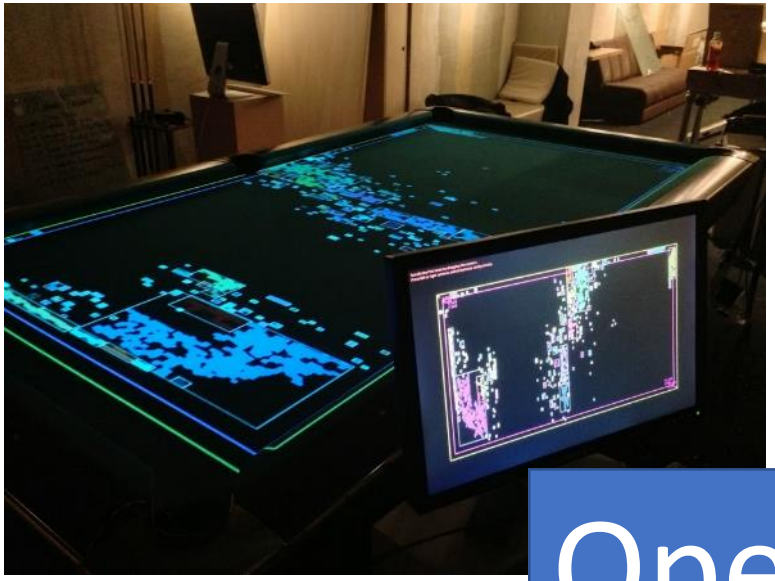
なめこカメラ

おうちハック発表会

8/31@秋葉原Garage

参加者募集中！

http://kadecot.net/ouch-hack/





OpenPool

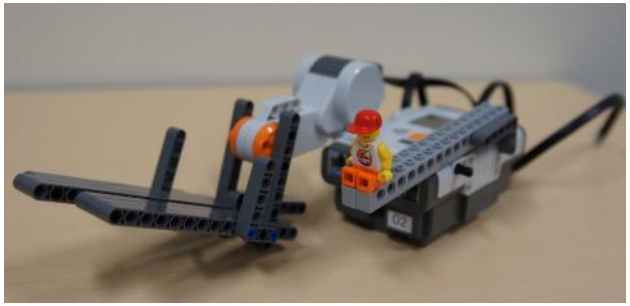






Kinect + LEGO Workshop









実世界Programmingは難しい

実世界入出力を伴うプログラムの増加

実世界入力
（センサの生データ）

a. RGBカメラ → 映像情報

b. 深度センサ → 姿勢情報





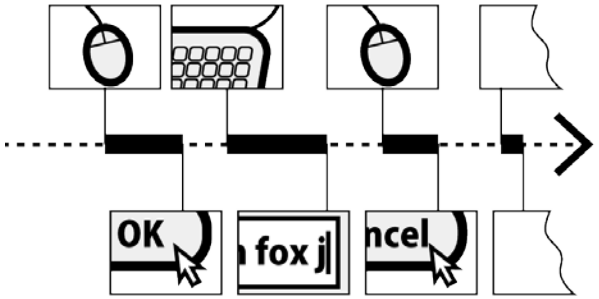
実世界出力
（アクチュエータの生データ）

c. 姿勢情報 → ロボット

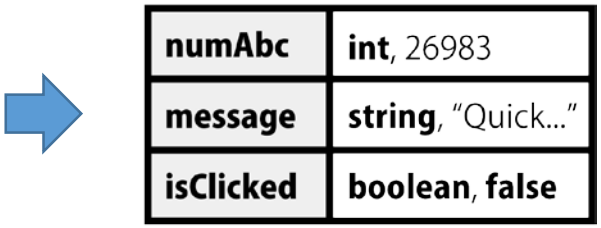


実世界入出力を伴うプログラム開発の特徴

従来型入出力:

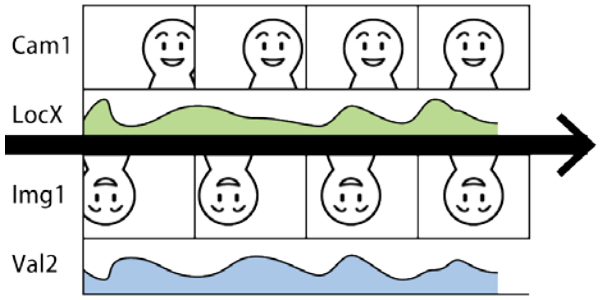


シンプルで断続的

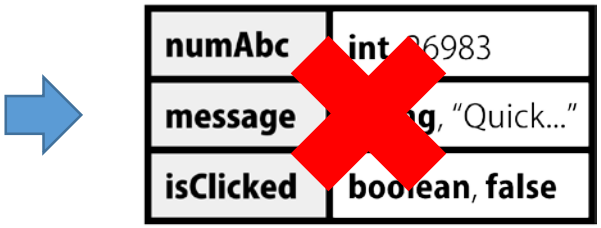


文字表記が容易

実世界入出力:



複雑で連続的



文字表記が困難

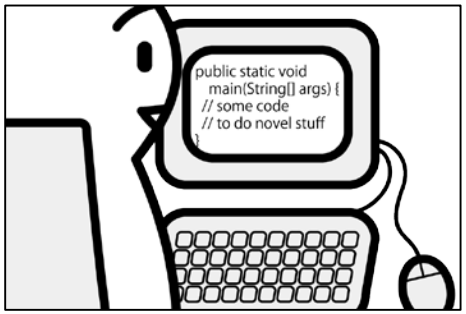
実世界入出力を伴うプログラム開発の問題

既存の統合開発環境は、従来型の入出力を伴うプログラム開発に最適化されている

↓

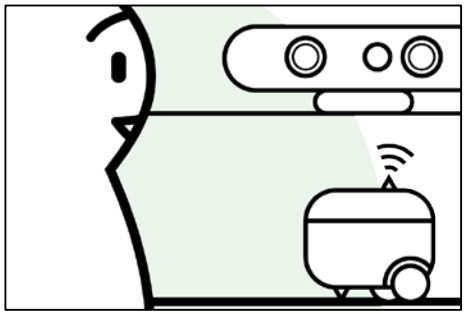
文字ベースのユーザインタフェースが多い

↓



開発環境（**2次元**）

実行結果を理解・
想像するのが困難



実行環境（**3次元**）

3次元への想像力

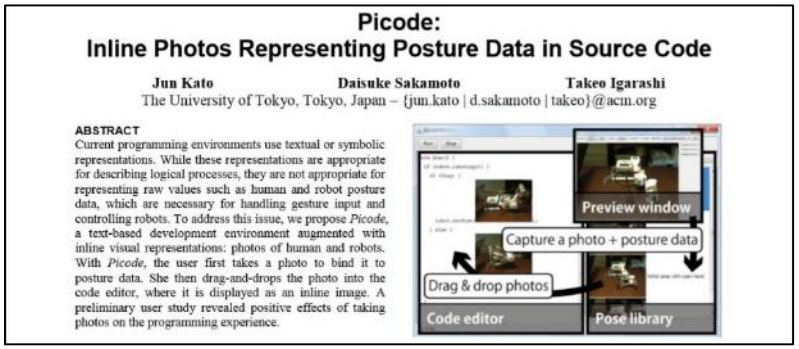
博士論文の内容:
Integrated Graphical Representations

文字表現 抽象的で精密なロジックを表現できる

画像表現 具体例を分かりやすく表現できる

Cf. 科学論文における
文章と図表の関係

"Orality and Literacy" [Ong '82]



Picode:
Inline Photos Representing Posture Data in Source Code

Jun Kato **Daisuke Sakamoto** **Takeo Igarashi**
The University of Tokyo, Tokyo, Japan – {jun.kato | d.sakamoto | takeo}@acm.org

ABSTRACT
Current programming environments use textual or symbolic representations. While these representations are appropriate for describing logical processes, they are not appropriate for representing raw values such as human and robot posture data, which are necessary for handling gesture input and controlling robots. To address this issue, we propose *Picode*, a text-based development environment augmented with inline visual representations: photos of human and robots. With *Picode*, the user first takes a photo to bind it to posture data. She then drag-and-drops the photo into the code editor, where it is displayed as an inline image. A preliminary user study revealed positive effects of taking photos on the programming experience.

➡ 実世界由来のデータを画像表現で表し、
文字プログラミングの効率向上をねらう

実世界入出力データを表す画像表現:

写真

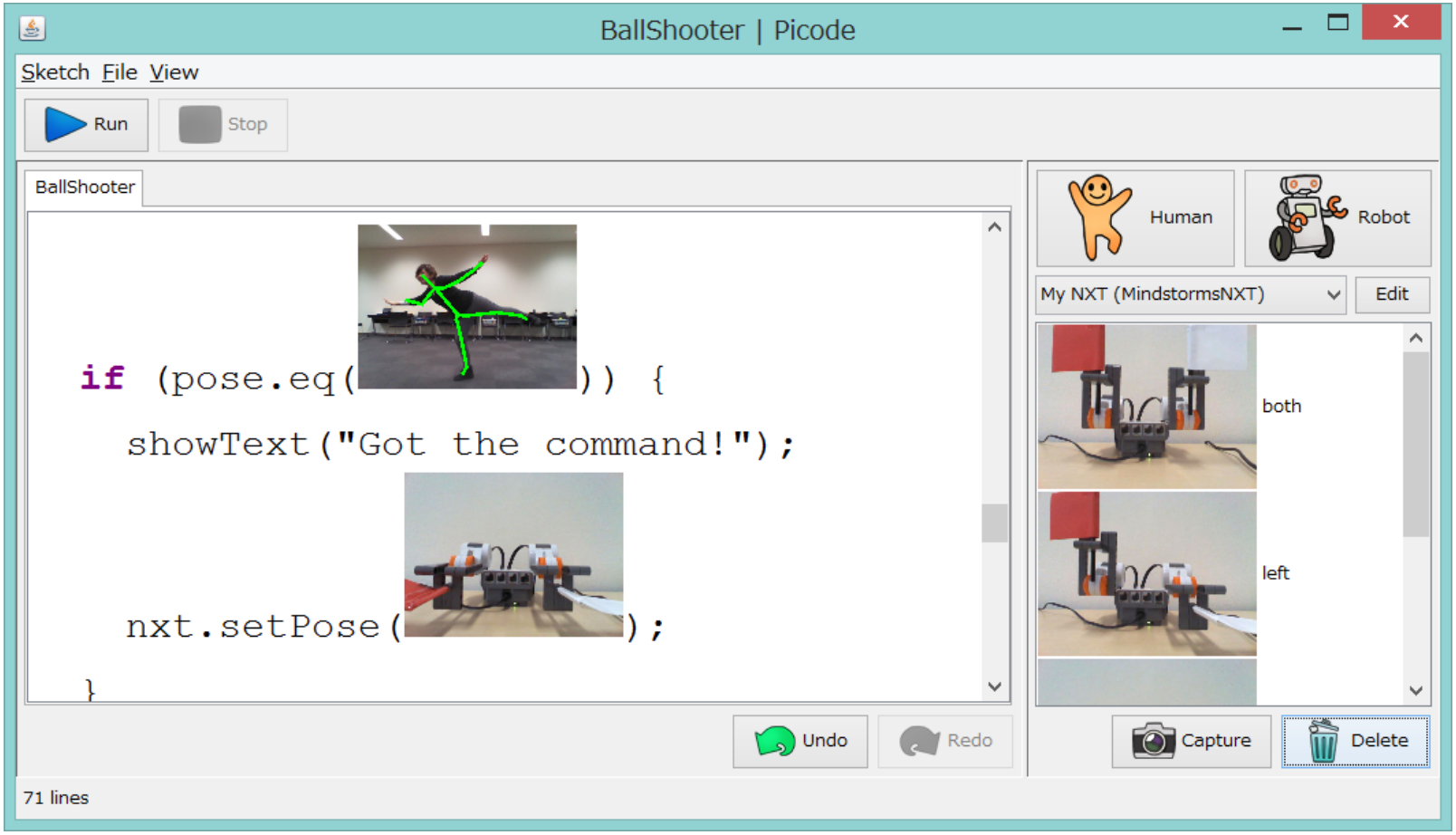
- 実世界入出力データを写真と紐づけて管理
- 文字列ベースのエディタ中でインライン表示

HumanPose pose = 

;

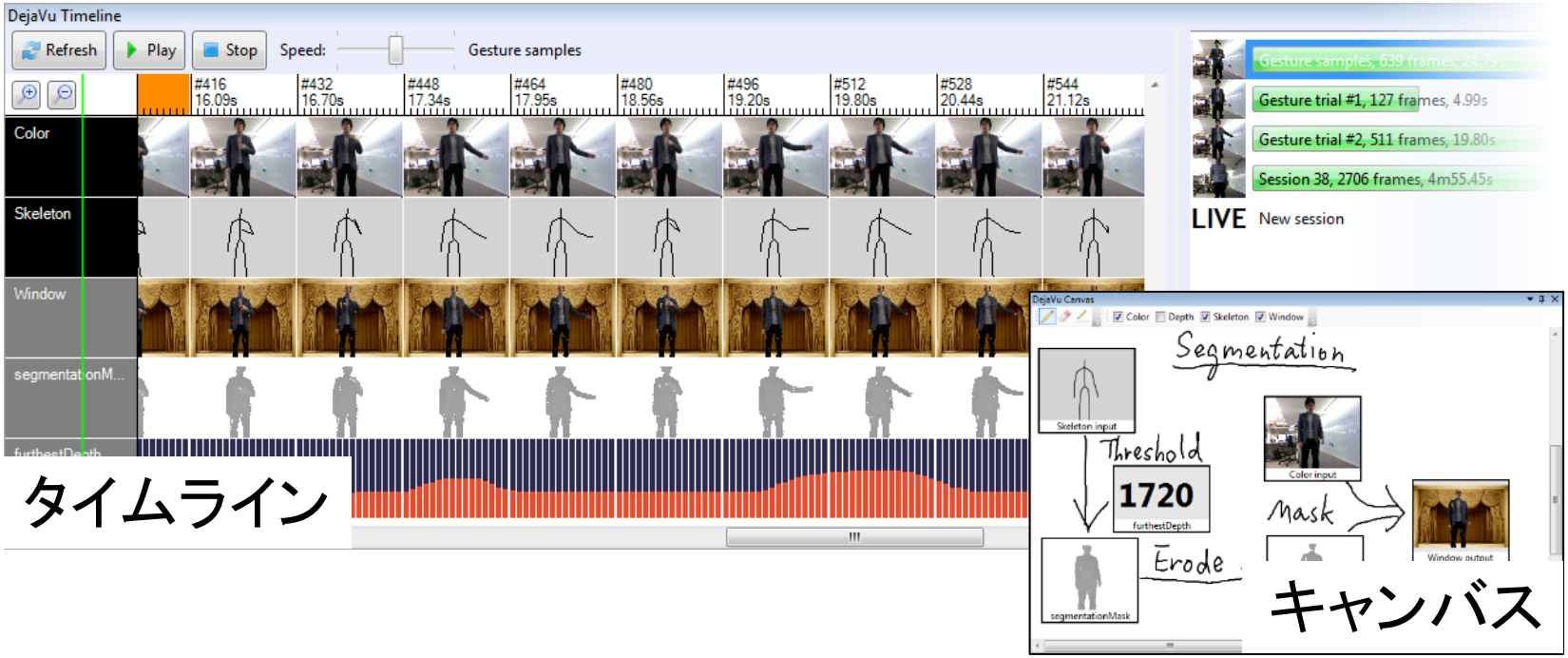
試作システム: Picode

[Kato CHI ’13]



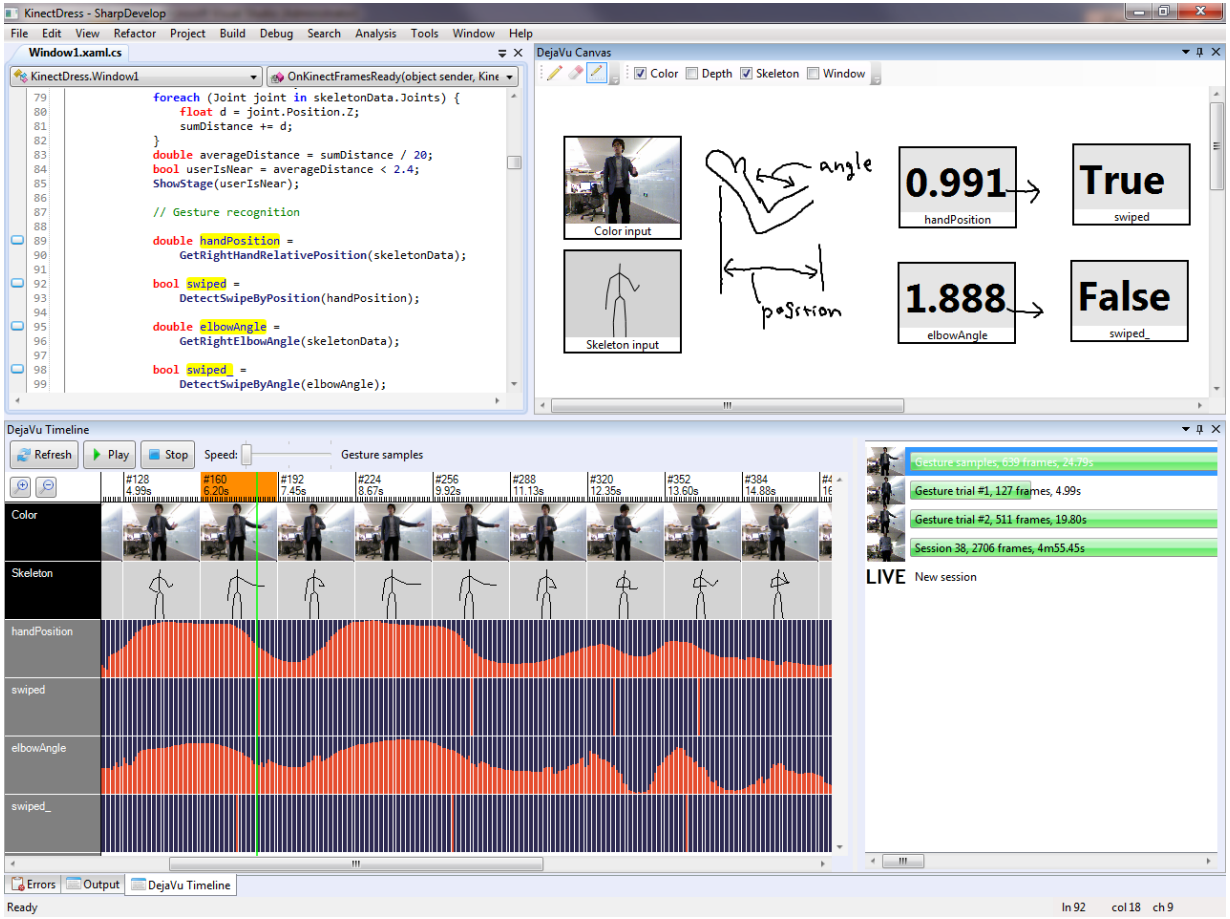
実世界入出力データを表す画像表現:
動画

- プログラムの振る舞いを動画として記録・管理
- 2つのユーザインタフェースがコードエディタと連携

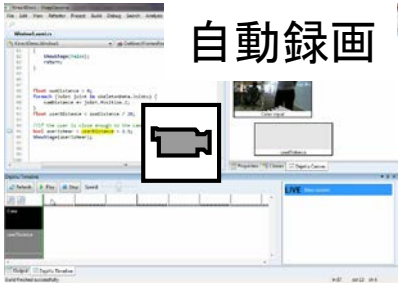


試作システム:
DejaVu

[Kato UIST ’12]



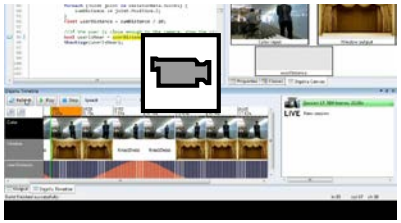
自動録画



録画データの再生



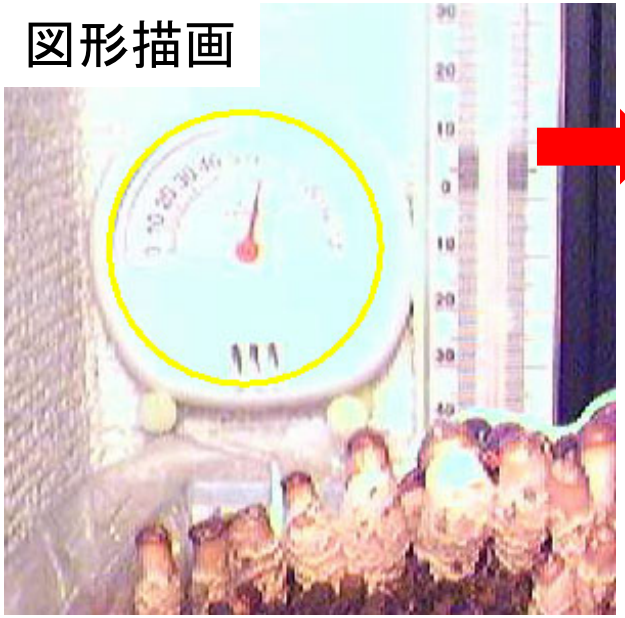
プログラム再実行



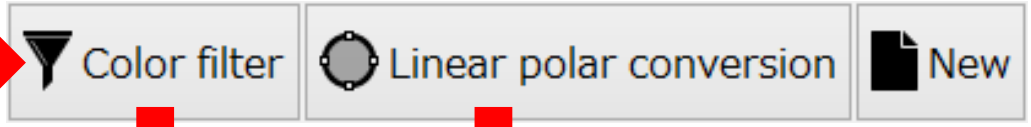
画像表現の編集操作による プログラム実装支援

- Example（直前の実行結果）への描画で実装を行う
- 結果をインタラクティブに確認

図形描画



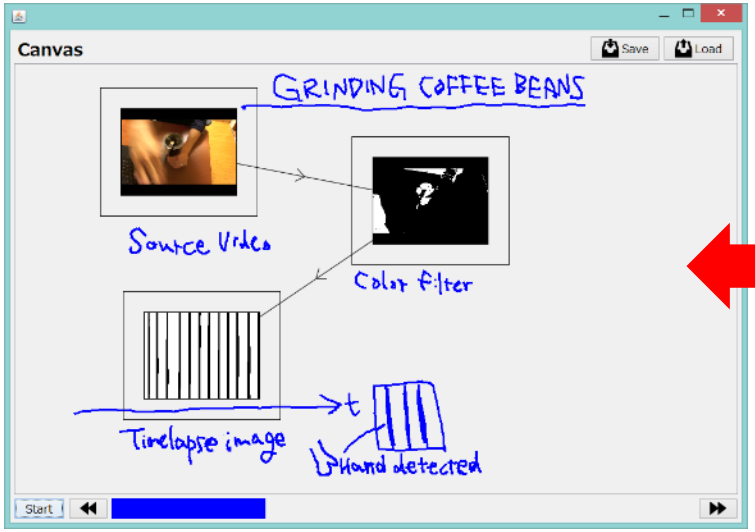
コード補完

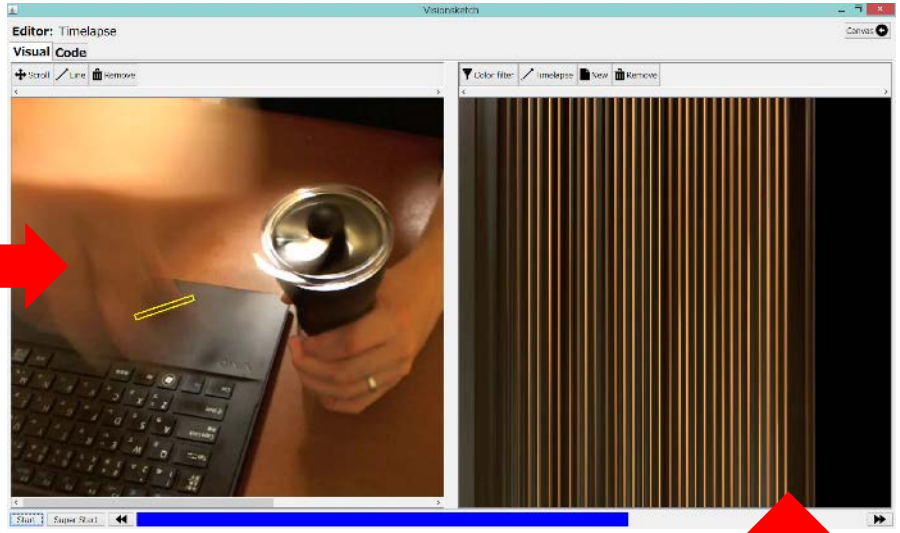




結果をインタラクティブに確認

試作システム: VisionSketch





Canvas

Visual Editor

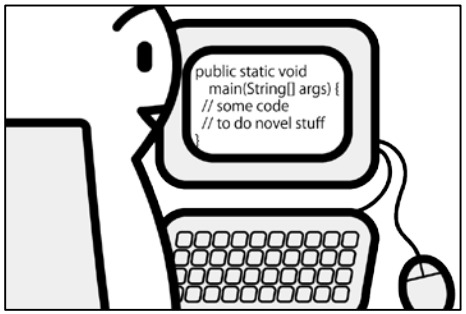


Code Editor

3つの研究のまとめ:

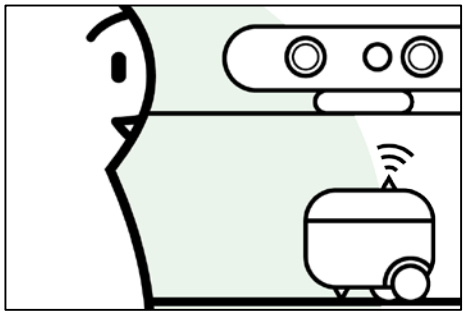
開発環境への画像表現の統合

1. 実世界入出力を伴うプログラム開発のワークフロー（Programming with Example, PwE）を分析
2. 画像表現を活用しPwEを支援する開発環境試作



開発環境（**2次元**）

溝を少し埋めた



実行環境（**3次元**）

Live Programming

プログラムは死んでいる

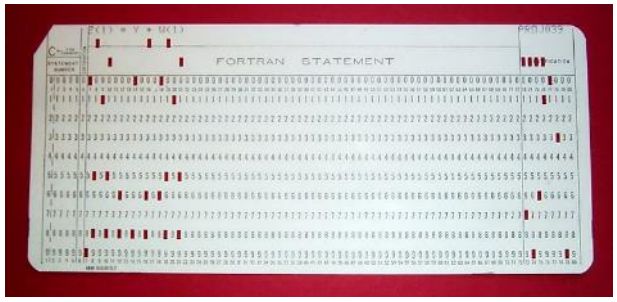
プログラミング今昔 (1/3)

- コンピュータに、やってほしい手続きを伝える方法
- かつてのプログラミング [～1960年代前半]



記録する内容は
自力で計算

機械語を**カード**に記録



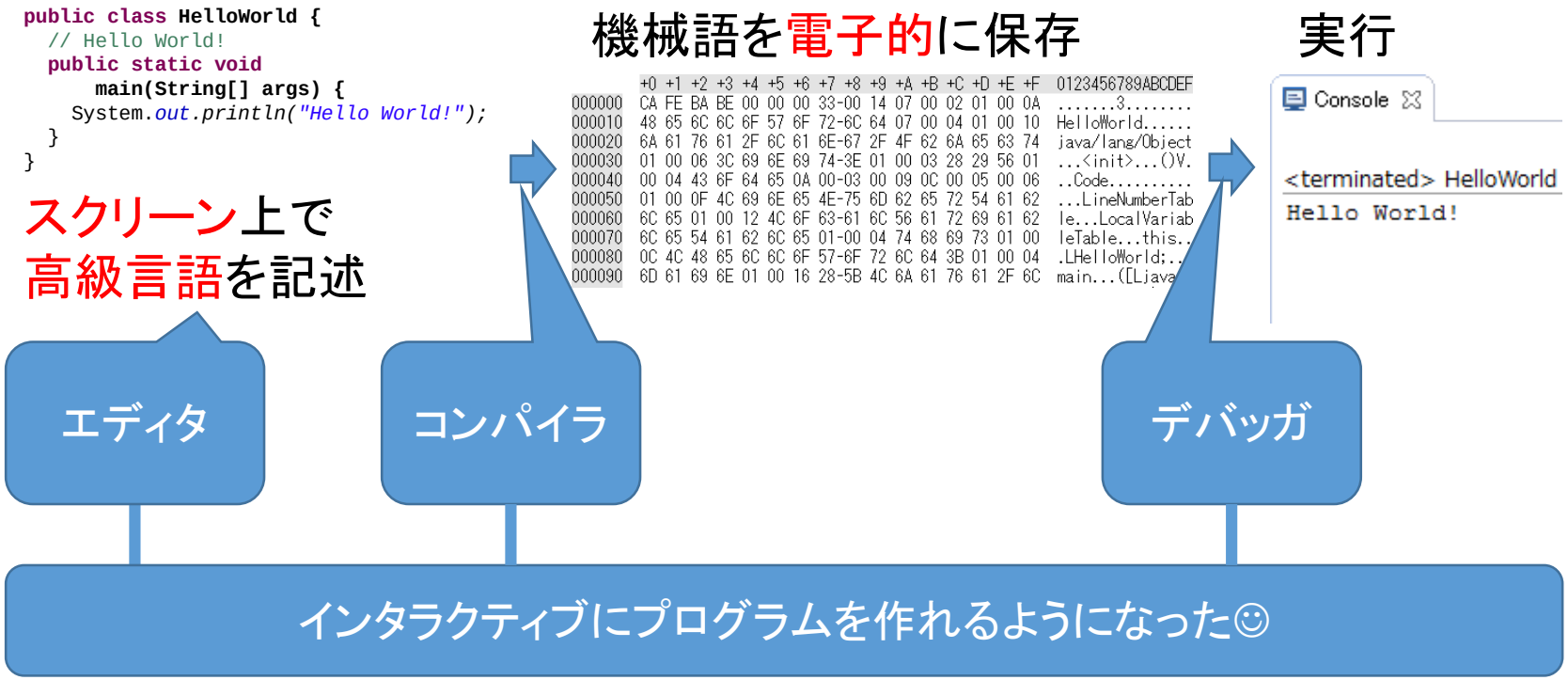
実行



プログラミングは全くインタラクティブじゃなかった☹

プログラミング今昔 (2/3)

- その後のプログラミング [1960年代後半～]



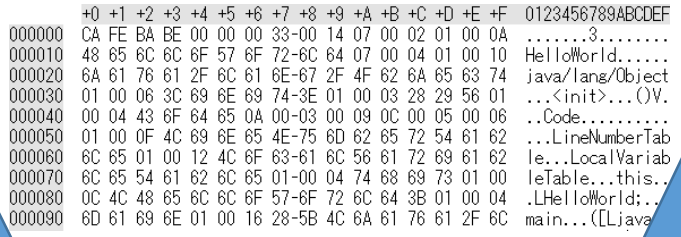
プログラミング今昔 (2/3)

```
public class HelloWorld {
  // Hello World!
  public static void
      main(String[] args) {
    System.out.println("Hello World!");
  }
}
```

スクリーン上で
高級言語を記述

エディタ

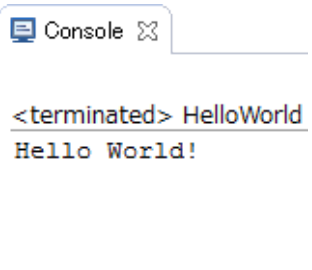
機械語を電子的に保存



```
        +0 +1 +2 +3 +4 +5 +6 +7 +8 +9 +A +B +C +D +E +F  0123456789ABCDEF
000000  CA FE BA BE 00 00 00 33-00 14 07 00 02 01 00 0A  .......3........
000010  48 65 6C 6C 6F 57 6F 72-6C 64 07 00 04 01 00 10  HelloWorld......
000020  6A 61 76 61 2F 6C 61 6E-67 2F 4F 62 6A 65 63 74  java/lang/Object
000030  01 00 06 3C 69 6E 69 74-3E 01 00 03 28 29 56 01  ...<init>...()V.
000040  00 04 43 6F 64 65 0A 00-03 00 09 0C 00 05 00 06  ..Code..........
000050  01 00 0F 4C 69 6E 65 4E-75 6D 62 65 72 54 61 62  ...LineNumberTab
000060  6C 65 01 00 12 4C 6F 63-61 6C 56 61 72 69 61 62  le...LocalVariab
000070  6C 65 54 61 62 6C 65 01-00 04 74 68 69 73 01 00  leTable...this..
000080  0C 4C 48 65 6C 6C 6F 57-6F 72 6C 64 3B 01 00 04  .LHelloWorld;...
000090  6D 61 69 6E 01 00 16 28-5B 4C 6A 61 76 61 2F 6C  main...([Ljava
```

コンパイラ

実行



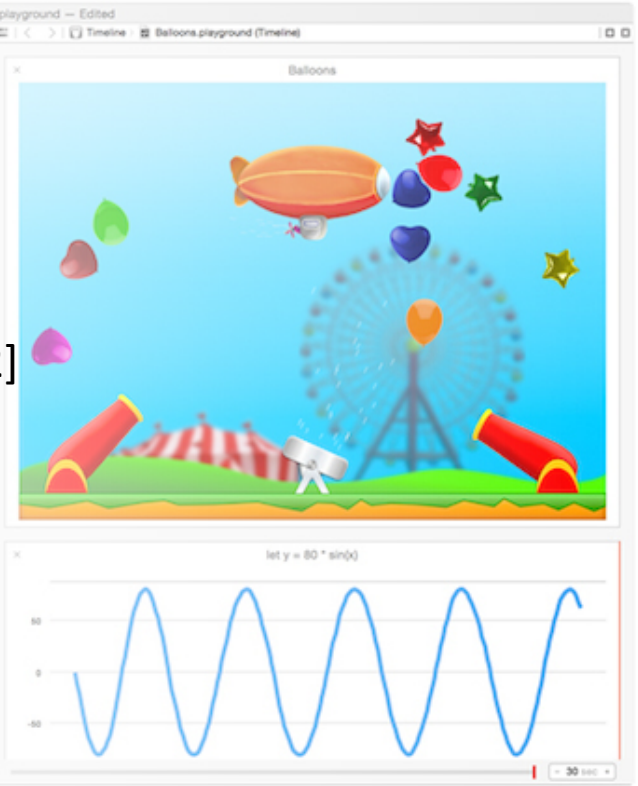
デバッガ

依然としてこれらのステップは厳密に別れている

ここにも **溝** があった！

Live Programming

- Inventing on Principles [Victor, 2012]
- Light Table [2012]
- TouchDevelop [PLDI 2013]
- Swift + Xcode [2014]



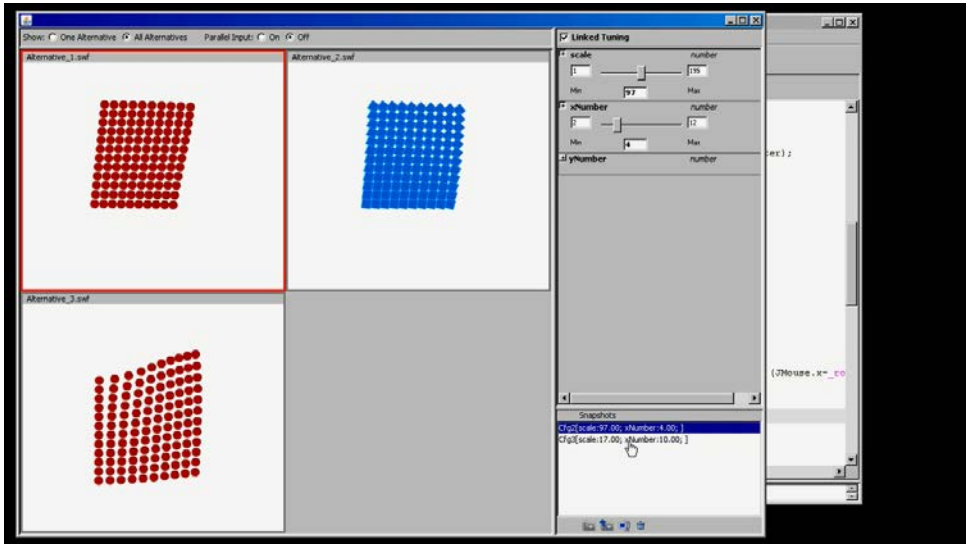
溝 を埋める試み

静的表現 (ソースコード)　**動的表現** (実行時の振る舞い)

プログラムを生かす

Live Programming

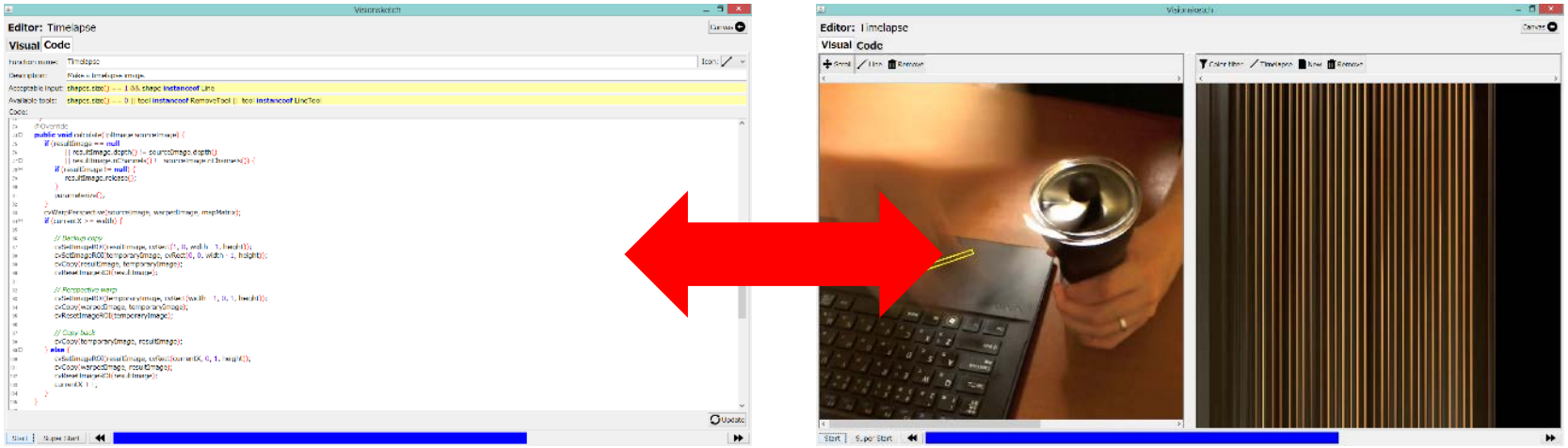
- （定義1）プログラミングの間ずっと、プログラムの現状に関する情報が常に与えられる開発環境
- （同2）コンパイル・実行等の溝をなくす開発環境



Juxtapose
[Hartmann, UIST 2008]

VisionSketch

- プログラム全体を再起動せず、気になっているところだけ書き換えて結果を確認できる
- アプリケーションを限定することで、
  Hot Swapを強力にしたLive Programming環境



Live Programmingの本質

プログラム＝

- ソースコード＋実行時の動的振る舞い
- 不揮発性記憶媒体（HDD,SSD）＋揮発性（メモリ）
- 関数＋状態情報



けっきょく、どちらも{0,1}+の **情報** であり、区別は便宜的



区別をなくすと便利じゃない？

生かしたまま
壊していいところを
明示する

Morphic的世界観への回帰？

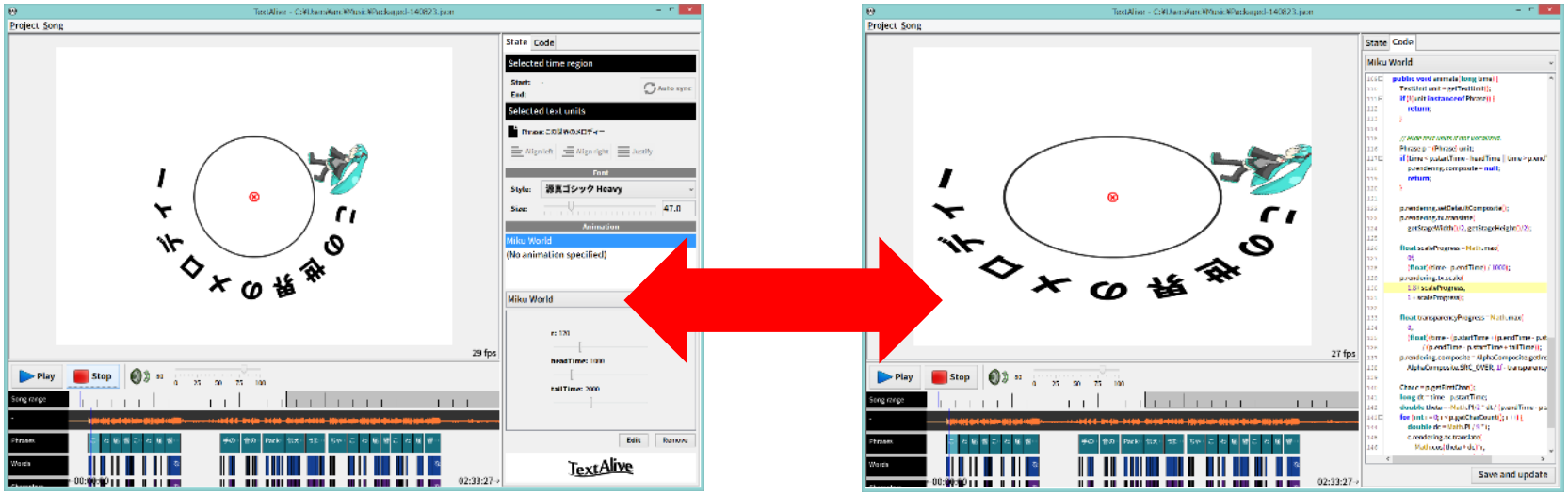
- Morphic
  - Self, Squeak用GUIツールキット
  - 見えているものは全部編集可能
  - http://lively-kernel.org/



- 見えているものすべてを編集可能な道具は人類には早すぎるのでは？（自由度が高い＝脆弱）
  - Atomはテキストエディタに特化したらから実用的
  - VisionSketchは画像処理に特化したから実用的
  - …

TextAlive

- Kinetic Typography動画を簡単に生成できるコンテンツ制作環境
- 動画生成過程に使うアルゴリズムをLive Programmingできる



実世界Live Programmingの実現に向けて

まとめ

- 実世界Programming
  - 2次元と3次元の溝
  - 文字表現だけでは無理がある
  - 開発環境には分かりやすいメディア表現が必要
- Live Programming
  - 記述と実行の溝
  - 実行したまま編集したい
  - すべてを編集可能にした環境は脆弱すぎる
  - 開発環境はもっとファンシーになるべき